LAPORAN PRAKTIKUM

STRUKTUR DATA

“Queue”

disusun oleh: Dennis Shauqi Akbar

2511533020

Dosen pengampu: Dr. Wahyudi. S.T.M.T

Asisten Pratikum: Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS TAHUN

2025

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman. Tugas Individu ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Struktur Data, dengan Judul "Queue"

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pengampu yang telah memberikan arahan sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami.

Padang, 30 April 2026

Dennis Shauqi Akbar

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan 1

1.3 Manfaat 2

BAB II PEMBAHASAN1 11

1.1 Pengertian Queue 11

2.2 Implementasi Pada Java 11

2.2.1 IterasiQueue_3020 11

2.2.2 QueueArray_2511533020 5

2.2.3 QueueArrayDriver_2511533020 5

2.2.4 QueueLinkList_2511533020 6

2.2.5 ReverseData_2511533020 6

TUGAS PRAKTIKUM 11

BAB I
PENDAHULUAN

1.1 Latar Belakang

Queue atau antrian adalah struktur data linear yang bekerja berdasarkan prinsip FIFO (First In First Out), artinya elemen yang pertama masuk akan menjadi elemen yang pertama keluar. Prinsip ini sangat umum ditemui dalam kehidupan sehari-hari, seperti antrian di loket bank, antrian pembayaran kasir, atau antrian kendaraan di pintu tol. Dalam dunia pemrograman, Queue digunakan untuk berbagai keperluan seperti manajemen proses pada sistem operasi (scheduling), penanganan permintaan pada server, serta pada algoritma pencarian BFS (Breadth First Search).

Berbeda dengan Stack yang menggunakan prinsip LIFO, Queue memiliki dua ujung yang berbeda: ujung depan (front) untuk penghapusan elemen, dan ujung belakang (rear) untuk penambahan elemen. Operasi utama pada Queue adalah enqueue (menambah elemen ke belakang) dan dequeue (menghapus elemen dari depan). Selain itu terdapat operasi peek atau front untuk melihat elemen depan tanpa menghapus, serta isEmpty untuk mengecek apakah Queue kosong.

Praktikum ini menggunakan bahasa Java dengan berbagai implementasi Queue, mulai dari penggunaan Queue bawaan Java (dengan LinkedList), implementasi Queue menggunakan array sirkular, hingga pemanfaatan Queue dengan bantuan Stack untuk membalik data. Melalui kelima program yang diberikan, mahasiswa diharapkan mampu memahami konsep FIFO, membedakan berbagai cara implementasi Queue, serta mampu menerapkannya dalam kasus-kasus sederhana.

1.2 Tujuan

Tujuan dari praktikum ini adalah memahami konsep dasar Queue dan prinsip FIFO (First In First Out) melalui implementasi dalam bahasa Java. Mahasiswa diharapkan mampu mengoperasikan method-method utama Queue yaitu add/offer (enqueue), remove/poll (dequeue), peek (melihat elemen depan), serta isEmpty dan size. Selain itu, praktikum ini bertujuan untuk membedakan implementasi Queue menggunakan kelas bawaan Java (Queue dengan LinkedList), menggunakan array sirkular (circular array), serta menggunakan LinkedList secara langsung

Selanjutnya, melalui kelima kode program yang diberikan (mulai dari IterasiQueue_3020, QueueArray_2511533020, QueueArrayDriver_2511533020, QueueLinkList_2511533020, hingga ReverseData_2511533020), mahasiswa dapat mengamati bagaimana Queue digunakan untuk menyimpan data dengan urutan FIFO, bagaimana menangani kondisi penuh dan kosong pada implementasi array sirkular, serta bagaimana memanfaatkan Stack untuk membalik urutan data dalam Queue. Pada akhir praktikum, mahasiswa diharapkan dapat memilih implementasi Queue yang tepat sesuai dengan kebutuhan program.

1.3 Manfaat

Manfaat yang diperoleh dari praktikum ini adalah penguasaan terhadap struktur data Queue yang sangat berguna dalam pengembangan perangkat lunak sehari-hari, seperti pada sistem antrian pelanggan, manajemen tugas (task scheduling), atau buffer pada komunikasi data. Dengan memahami Queue, mahasiswa dapat membuat program yang memproses data berdasarkan urutan kedatangan, seperti simulasi antrian di bank atau sistem pemesanan tiket.

Selain itu, praktikum ini memberikan manfaat langsung dalam hal pemahaman tentang implementasi Queue menggunakan array sirkular. Mahasiswa belajar bagaimana memanfaatkan operasi modulo (%) untuk menggerakkan pointer front dan rear secara melingkar, sehingga kapasitas array dapat digunakan secara efisien.

BAB II
PEMBAHASAN1

1.1 Pengertian Queue

Queue (antrian) adalah struktur data linear yang mengikuti aturan FIFO (First In First Out), artinya elemen yang pertama kali dimasukkan akan menjadi elemen yang pertama kali dikeluarkan. Queue memiliki dua ujung yaitu ujung depan (front) untuk operasi penghapusan (dequeue) dan ujung belakang (rear) untuk operasi penambahan (enqueue). Operasi dasar Queue meliputi enqueue (menambah elemen), dequeue (menghapus elemen dari depan), front atau peek (melihat elemen depan tanpa menghapus), isEmpty (mengecek apakah Queue kosong), dan isFull (mengecek apakah Queue penuh, khusus untuk implementasi dengan array statis).

Queue dapat diimplementasikan menggunakan berbagai cara, antara lain menggunakan kelas bawaan Java (Queue dengan LinkedList atau ArrayDeque), menggunakan array statis dengan konsep sirkular (circular array), atau menggunakan linked list. Implementasi dengan array sirkular memerlukan operasi modulo (%) untuk menghitung posisi front dan rear agar array dapat digunakan berulang kali tanpa perlu menggeser elemen. Keuntungan Queue adalah kesederhanaan dan kepatuhan terhadap urutan kedatangan, sementara kelemahannya adalah akses acak tidak diperbolehkan (hanya depan dan belakang yang dapat diakses).

2.2 Implementasi Pada Java

Berikut adalah penjelasan untuk masing-masing kode program Stack yang telah dipraktikkan.

2.2.1 IterasiQueue_3020

```java
package pekan4_2511533020;
import java.util.*;

public class ReverseData_2511533020 {
    public static void main(String[] args) {
        Queue<Integer> q_3020 = new LinkedList<Integer>();
        q_3020.add(1);
        q_3020.add(2);
        q_3020.add(3);
        System.out.println("sebelum reverse" + q_3020);
        Stack<Integer> s_3020 = new Stack<Integer>();
        while(!q_3020.isEmpty()) { //Q -> S
            s_3020.push(q_3020.remove());
        }
        while (!s_3020.isEmpty()) {
            q_3020.add(s_3020.pop());

        }
        System.out.println("sesudah reverse= " + q_3020);
    }
}
```

(Gambar 2.2.1.1)

Program ini mendemonstrasikan penggunaan kelas Queue bawaan Java dengan implementasi LinkedList untuk menyimpan data String. Sebuah bjek Queue<String> bernama q_3020 dibuat menggunakan new LinkedList<>(). Method add() digunakan untuk menambahkan lima kata yaitu "Praktikum", "Struktur", "Data", "Dan", "Algoritma" secara berurutan. Program kemudian menggunakan Iterator untuk melakukan iterasi terhadap elemen-elemen dalam Queue. Iterator diperoleh dengan memanggil q_3020.iterator(), lalu perulangan while dilakukan selama hasNext() bernilai true. Di dalam perulangan, iterator.next() mengembalikan elemen berikutnya, lalu dicetak dengan spasi. Program ini menunjukkan cara mengakses seluruh elemen Queue tanpa menghapusnya, yaitu dengan bantuan Iterator.

Hasil output yang dihasilkan adalah “Praktikum Struktur Data Dan Algoritma.”

```
<terminated> IterasiQueue_3020 [Java Application] C:\Users\
Praktikum Struktur Data Dan Algoritma
```

(Gambar 2.2.1.2)

2.2.2 QueueArray_2511533020

```
    int front_3020() {
        if (isEmpty(this))
            return Integer.MIN_VALUE;

        return this.array_3020[this.front_3020];
    }
    int rear_3020() {
        if (isEmpty(this))
            return Integer.MIN_VALUE;
        return this.array_3020[this.rear_3020];

    }
    //mencetak elemen antrian
    void display_3020() {
        int i_3020;
        if (front_3020 == rear_3020) {
            System.out.printf("\nAntrian Kosong\n");
            return;

        }
        //kunjungi dari belakang dan cetak
        for (i_3020 = front_3020; i_3020 < rear_3020; i_3020++) {
            System.out.printf(" %d <-- ", array_3020[i_3020]);
        }
        return;
    }
}
```

(Gambar 2.2.2)

Kelas ini mengimplementasikan Queue menggunakan array sirkular (circular array). Atribut yang digunakan meliputi front_3020 (indeks depan), rear_3020 (indeks belakang), size_3020 (jumlah elemen saat ini), capacity_3020 (kapasitas maksimal array), serta array_3020 (array penyimpan data bertipe int). Konstruktor menerima parameter capacity_3020, menginisialisasi front_3020 = 0, size_3020 = 0, dan rear_3020 = capacity_3020 - 1.

Method isFull() mengecek apakah size_3020 == capacity_3020, sedangkan isEmpty() mengecek size_3020 == 0. Method enqueue_3020(int item_3020) menambah elemen dengan cara: rear_3020 = (rear_3020 + 1) % capacity_3020, lalu nilai dimasukkan ke array, dan size_3020 ditambah. Penggunaan modulo memungkinkan rear kembali ke indeks 0 setelah mencapai ujung array.

Method dequeue_3020() mengambil elemen dari depan dengan cara: mengambil nilai dari array_3020[front_3020], lalu front_3020 = (front_3020 + 1) % capacity_3020, dan size_3020 dikurangi. Method front_3020() dan rear_3020() mengembalikan elemen depan dan belakang tanpa menghapus. Method display_3020() mencetak elemen dari front hingga rear. Kelas ini tidak memiliki method main sendiri, melainkan diuji oleh driver terpisah.

2.2.3 QueueArrayDriver_2511533020

```java
package pekan4_2511533020;

public class QueueArrayDriver_2511533020 {
        public static void main(String[] args) {
            QueueArray_2511533020 queue_3020 = new QueueArray_2511533020(1000) ;
            queue_3020.enqueue_3020(10);
            queue_3020.enqueue_3020(20);
            queue_3020.enqueue_3020(30);
            queue_3020.enqueue_3020(40);
            System.out.println("Item di depan" + queue_3020.front_3020());
            System.out.println("Item paling belakang" + queue_3020.rear_3020());
            System.out.println("tampilan queue");
            queue_3020.display_3020();
            System.out.println(queue_3020.dequeue_3020() + " dihapus dari queue");
            System.out.println("item di depan: " + queue_3020.front_3020());
            System.out.println("item di belakang: " + queue_3020.rear_3020());
            System.out.println("tampilan queue setelah satu data dihapus");
            queue_3020.display_3020();


        }
}
```

(Gambar 2.2.3)

Driver ini menguji kelas QueueArray_2511533020. Sebuah objek queue_3020 dibuat dengan kapasitas 1000. Kemudian dilakukan enqueue_3020 untuk nilai 10, 20, 30, 40 secara berurutan. Program mencetak elemen depan menggunakan front_3020() (hasilnya 10) dan elemen belakang menggunakan rear_3020() (hasilnya 40). Method display_3020() dipanggil untuk mencetak seluruh elemen. Selanjutnya, dequeue_3020() dipanggil yang mengembalikan 10, lalu dicetak bahwa 10 dihapus dari queue.

Setelah penghapusan, front_3020() mengembalikan 20, dan rear_3020() tetap 40. display_3020() dipanggil lagi untuk menunjukkan isi queue setelah penghapusan. Output yang dihasilkan menunjukkan bahwa Queue bekerja sesuai prinsip FIFO: elemen yang pertama masuk (10) menjadi yang pertama keluar. Program ini juga mendemonstrasikan penggunaan array sirkular dengan operasi modulo.

2.2.4 QueueLinkList_2511533020

```java
package pekan4_2511533020;

import java.util.*;

public class QueueLinkList_2511533020 {

    public static void main(String[] args) {
        Queue<Integer> q_3020 = new LinkedList<>() ;
        // tambah elemen {0, 1, 2, 3, 4, 5} ke antrian

        for (int i_3020 = 0; i_3020 < 6; i_3020++)
            q_3020.add(i_3020);
            //Menampilkan isi antrian.
            System.out.println("Elemen antrian " + q_3020);
            //Untuk menghapus kepala antrian.
            int hapus_3020 = q_3020.remove();
            System.out.println(q_3020);
            //Untuk melihat antrian terdepan
            int depan_3020 = q_3020.peek();
            System.out.println("Kepala Antrian = " + depan_3020);

            int banyak_3020 = q_3020.size();
            System.out.println("Size Antrian = " + banyak_3020);
        }



}
```

(Gambar 2.2.4)

Program ini menggunakan Queue bawaan Java dengan implementasi LinkedList untuk menyimpan data Integer. Sebuah objek Queue<Integer> bernama q_3020 dibuat. Perulangan for dari 0 hingga 5 digunakan untuk menambahkan angka 0,1,2,3,4,5 ke dalam Queue menggunakan method add(). Program kemudian mencetak isi Queue dengan System.out.println(q_3020), yang menghasilkan [0, 1, 2, 3, 4, 5]. Method remove() digunakan untuk menghapus kepala antrian (elemen pertama), yaitu angka 0, lalu Queue dicetak lagi menjadi [1, 2, 3, 4, 5].

Method peek() digunakan untuk melihat elemen depan tanpa menghapusnya, hasilnya adalah 1. Method size() mengembalikan jumlah elemen dalam Queue, yaitu 5. Program ini menunjukkan operasi dasar Queue menggunakan kelas bawaan Java, serta perbedaan antara remove() (menghapus dan melempar exception jika kosong) dan peek() (melihat tanpa menghapus).

2.2.5 ReverseData_2511533020

```java
package pekan4_2511533020;
import java.util.*;

public class ReverseData_2511533020 {
    public static void main(String[] args) {
        Queue<Integer> q_3020 = new LinkedList<Integer>();
        q_3020.add(1);
        q_3020.add(2);
        q_3020.add(3);
        System.out.println("sebelum reverse" + q_3020);
        Stack<Integer> s_3020 = new Stack<Integer>();
        while(!q_3020.isEmpty()) { //Q -> S
            s_3020.push(q_3020.remove());
        }
        while (!s_3020.isEmpty()) {
            q_3020.add(s_3020.pop());

        }
        System.out.println("sesudah reverse= " + q_3020);
    }
}
```

(Gambar 2.2.5.1)

Program ini mendemonstrasikan bagaimana membalik urutan data dalam sebuah Queue dengan bantuan Stack. Queue q_3020 diinisialisasi dengan angka 1,2,3 menggunakan method add(). Isi Queue awal dicetak, hasilnya [1, 2, 3]. Prinsip FIFO membuat urutan ini tetap 1 (depan), 2, 3 (belakang). Untuk membalik, program membuat sebuah Stack<Integer>. Perulangan while selama Queue tidak kosong akan memindahkan setiap elemen dari Queue ke

Stack menggunakan s_3020.push(q_3020.remove()). Karena Stack bersifat LIFO, maka elemen terakhir dari Queue (3) akan masuk ke Stack terlebih dahulu? Perhatikan: Queue.remove() mengambil dari depan, sehingga urutan pemindahan adalah 1,2,3. Saat push ke Stack, 1 masuk duluan (paling bawah), lalu 2, lalu 3 (paling atas). Kemudian perulangan kedua memindahkan dari Stack kembali ke Queue menggunakan q_3020.add(s_3020.pop()). Karena pop mengambil dari atas Stack (LIFO), maka urutan yang keluar dari Stack adalah 3,2,1. Hasilnya Queue menjadi [3, 2, 1].

Program ini menunjukkan kombinasi dua struktur data (Queue dan Stack) untuk mengubah urutan data. Output yang dihasilkan adalah sebelum reverse[1, 2, 3] dan sesudah reverse= [3, 2, 1].

```
<terminated> ReverseData_2511533020 [Java Application] C:\Users\denni\.
sebelum reverse[1, 2, 3]
sesudah reverse= [3, 2, 1]
```

(Gambar 2.2.5.2)

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum Stack yang telah dilakukan, dapat disimpulkan bahwa Stack merupakan struktur data dengan prinsip LIFO yang dapat diimplementasikan menggunakan berbagai cara: menggunakan kelas Stack bawaan Java, menggunakan ArrayList, maupun menggunakan array statis. Kelima program yang diberikan berhasil menunjukkan operasi dasar push, pop, peek, serta penerapan Stack dalam studi kasus seperti mencari nilai maksimum tanpa merusak data dan mengevaluasi ekspresi postfix.

3.2 Saran

Untuk pengembangan lebih lanjut, disarankan agar implementasi Stack menggunakan array statis (stackArray_2511533020) ditambahkan kemampuan untuk mengubah ukuran array secara dinamis (resize) agar tidak mudah overflow. Pada program NilaiMaksimum_2511533020, sebaiknya ditambahkan penanganan jika stack kosong. Program SiswaStack_2511533020 dapat dilengkapi dengan method untuk mencari siswa berdasarkan nim. Selain itu, praktikum berikutnya dapat memperluas penggunaan Stack untuk konversi infix ke postfix atau untuk memeriksa keseimbangan tanda kurung pada ekspresi matematika. Seluruh program sudah memenuhi aturan penamaan dengan akhiran NIM 3020 pada atribut dan variabel lokal yang relevan.

3.3 Penutup

Demikian laporan praktikum Struktur Data dengan judul Stack ini disusun. Laporan ini mencakup penjelasan teoritis, implementasi kode, serta analisis dari lima program yang telah dibuat. Semoga laporan ini bermanfaat untuk memahami konsep Stack dan penerapannya dalam bahasa pemrograman Java. Penulis menyadari masih terdapat kekurangan, oleh karena itu kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa mendatang.

DAFTAR PUSTAKA

[1] Oracle, "Queue (Java Platform SE 17)," Oracle. [Online]. Available: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Queue.html (diakses: 30 April 2026).

[2] W3Schools, "Java Queue," W3Schools. [Online]. Available: https://www.w3schools.com/java/java_queue.asp (diakses: 30 April 2026).

[3] Wikipedia, "Queue (abstract data type)," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Queue_(abstract_data_type) (diakses: 30 April 2026).

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

TUGAS PRAKTIKUM

- **KELAS DRIVER:**

```
package pekan4;

public class AntrianLoket_2511533020 {
 private int front;
 private int rear;
 private int max;
 private String[] queue;
 private int size;

 public AntrianLoket_2511533020() {
 this.max = 100;
 this.queue = new String[max];
 this.front = 0;
 this.rear = -1;
 this.size = 0;
 }

 public boolean isEmpty() {
 return size == 0;
 }

 public boolean isFull() {
 return size == max;
 }

 public void enqueue(String nama) {
 if (isFull()) {
 System.out.println("Antrian penuh! Tidak dapat menambahkan pelanggan.");
 return;
 }
 rear = (rear + 1) % max;
 queue[rear] = nama;
 size++;
 System.out.println("Data berhasil ditambahkan ke antrian");
 }

 public void dequeue() {
 if (isEmpty()) {
 System.out.println("Antrian kosong! Tidak ada pelanggan yang dapat
dilayani.");
 return;
 }
 String namaTerlayani = queue[front];
 queue[front] = null;
 front = (front + 1) % max;
 size--;
 System.out.println(namaTerlayani + " telah dilayani");
 }

 public void display() {
```

```
    if (isEmpty()) {
    System.out.println("Antrian kosong!");
    return;
    }
    System.out.println("Isi antrian:");
    for (int i = 0; i < size; i++) {
    int index = (front + i) % max;
    System.out.println((i + 1) + ". " + queue[index]);
    }
    }

    public void reverse() {
    if (isEmpty()) {
    System.out.println("Antrian kosong! Tidak ada yang dapat dibalik.");
    return;
    }
    String[] tempArray = new String[size];
    for (int i = 0; i < size; i++) {
    tempArray[i] = queue[(front + i) % max];
    }
    for (int i = 0; i < size / 2; i++) {
    String temp = tempArray[i];
    tempArray[i] = tempArray[size - 1 - i];
    tempArray[size - 1 - i] = temp;
    }
    front = 0;
    rear = size - 1;
    for (int i = 0; i < size; i++) {
    queue[i] = tempArray[i];
    }
    System.out.println("Antrian berhasil dibalik!");
    }
  }
```

Kelas AntrianLoket_2511533020 merupakan implementasi Queue (antrian) menggunakan array sirkular. Kelas ini memiliki lima atribut: front (indeks depan untuk penghapusan), rear (indeks belakang untuk penambahan), max (kapasitas maksimal antrian yang ditentukan langsung di dalam kode yaitu 100), queue[] (array penyimpan data pelanggan bertipe String), dan size (jumlah elemen saat ini).

Konstruktor tanpa parameter menginisialisasi max = 100, lalu membuat array baru sebesar max, serta mengatur front = 0, rear = -1, dan size = 0. Method isEmpty() mengembalikan true jika size == 0, sedangkan isFull() mengembalikan true jika size == max. Method enqueue(String nama) menambahkan pelanggan ke antrian: jika penuh tampilkan pesan, jika tidak maka rear digerakkan secara circular dengan rumus (rear + 1) % max, data dimasukkan ke queue[rear], lalu size bertambah. Method dequeue() menghapus pelanggan dari depan: jika kosong tampilkan pesan, jika tidak maka nama di queue[front] disimpan, front digerakkan

dengan (front + 1) % max, size berkurang, lalu nama yang dihapus dicetak. Method display() menampilkan seluruh isi antrian dari depan ke belakang menggunakan perulangan dengan indeks (front + i) % max. Method reverse() membalik antrian dengan memindahkan data ke array sementara, membalik urutannya, lalu memasukkannya kembali ke queue[] dengan front = 0 dan rear = size - 1.

Seluruh method dalam kelas ini memastikan antrian bekerja sesuai prinsip FIFO (First In First Out).KELAS ADT

KELAS DRIVER

```java
package pekan4;

import java.util.*;

public class AntrianLoketDriver_2511533020 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        AntrianLoket_2511533020 antrian = new AntrianLoket_2511533020();
        int pilihan;

        System.out.println("=== PROGRAM ANTRIAN LOKET ===");

        do {
            System.out.println("\n1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama = input.nextLine();
                    antrian.enqueue(nama);
                    break;
                case 2:
                    antrian.dequeue();
                    break;
                case 3:
                    antrian.display();
                    break;
                case 4:
                    antrian.reverse();
                    break;
                case 5:
```

```java
                    System.out.println("Program selesai");
                    break;
                default:
                    System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan != 5);

        input.close();
    }
}
```

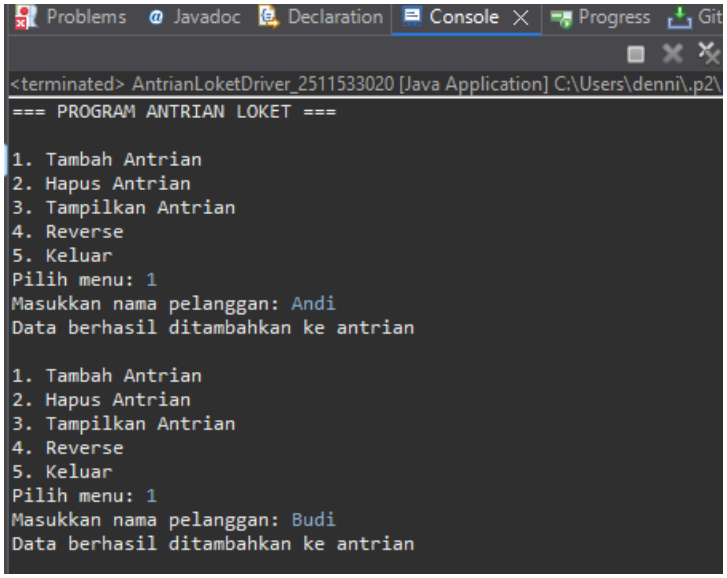
Kelas AntrianLoketDriver_2511533020 merupakan kelas driver yang berisi method main sebagai pusat kendali program. Pertama, program membuat objek AntrianLoket_2511533020

tanpa perlu memasukkan kapasitas karena kapasitas sudah ditentukan langsung di kelas ADT (default 100). Program kemudian menampilkan menu berulang menggunakan perulangan do-while selama pilihan tidak sama dengan 5.

Menu yang ditampilkan meliputi: 1. Tambah Antrian, 2. Hapus Antrian, 3. Tampilkan Antrian, 4. Reverse, dan 5. Keluar. Setiap pilihan menu diproses dengan struktur switch-case. Untuk pilihan 1, program meminta input nama pelanggan lalu memanggil method enqueue(). Pilihan 2 memanggil dequeue(). Pilihan 3 memanggil display(). Pilihan 4 memanggil reverse(). Pilihan 5 menampilkan pesan keluar dan menghentikan loop. Program juga menangani pilihan tidak valid dengan pesan default. Setelah loop selesai, Scanner ditutup untuk mencegah kebocoran memori.

Kelas driver ini memastikan bahwa semua fitur Queue dapat diakses dan diuji oleh pengguna secara interaktif sesuai dengan contoh output yang diminta dalam soal.

OUTPUT



```
<terminated> AntrianLoketDriver_2511533020 [Java Applicatio

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi
2. Budi

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik!

1. Tambah Antrian
2. Hapus Antrian
```

```
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Budi
2. Andi

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Budi telah dilayani

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
```

```
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```